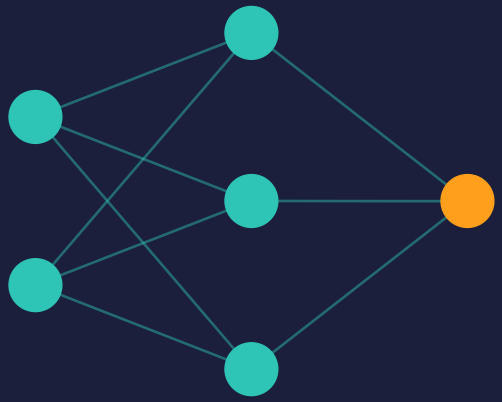
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۵

بهینه‌سازی: Momentum، Adam و زمان‌بندی نرخ یادگیری

بهینه‌سازهای مدرن، انتخاب نرخ یادگیری، Scheduler و مقداردهی اولیه وزن‌ها

کیوان محمودی

در این جلسه

۱ مشکلات SGD ساده

۲ Momentum

۳ RMSprop و نرخ یادگیری تطبیقی

۴ Adam و AdamW

۵ مقایسه روی Fashion-MNIST

۶ LR Range Test

۷ زمان‌بندی نرخ یادگیری

۸ مقداردهی اولیه: Xavier و He

چرا SGD ساده کافی نیست؟

۱ دره‌های باریک

انحنای متفاوت در جهت‌های مختلف: نوسان در جهت پرشیب و حرکت کند در جهت کم‌شیب.

۲ نویز گرادیان

گرادیان هر دسته فقط تخمینی از گرادیان واقعی است.

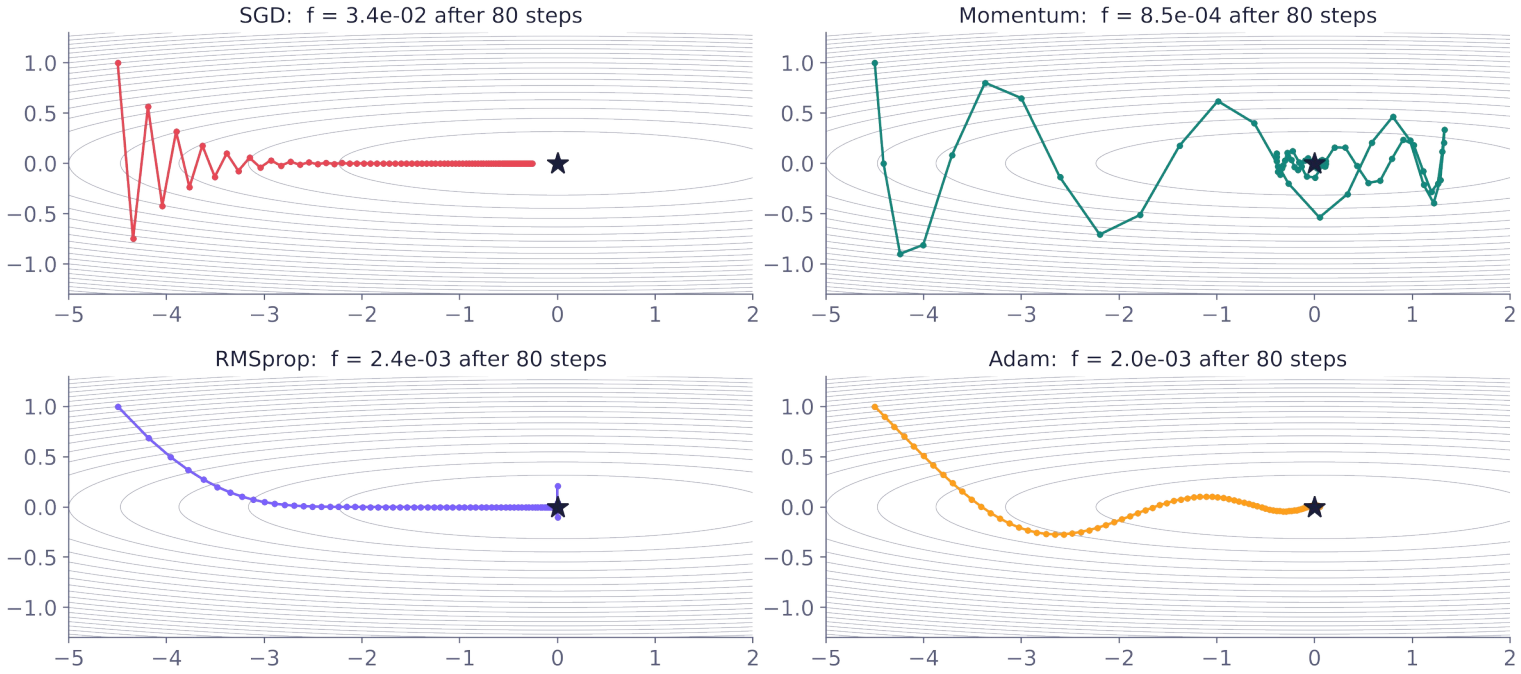
۳ نقاط زینی و فلات

در ابعاد بالا نقاطی با گرادیان نزدیک صفر که کمینه نیستند بسیار رایج‌اند.

۴ یک η برای همه

پارامترهایی با گرادیان کوچک خیلی کند یاد می‌گیرند.

چهار بهینه‌ساز در یک دره باریک



انحنا در جهت عمودی ۵۰ برابر جهت افقی است: $f = \frac{1}{2}(\theta_1^2 + 50\theta_2^2)$

Momentum: حافظه‌ای از گرادیان‌های قبلی

$$\mathbf{v}_t = \beta\, \mathbf{v}_{t-1} + \nabla \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta\, \mathbf{v}_t$$

- در جهت‌هایی که گرادیان ثابت است، سرعت جمع و حرکت تند می‌شود.
- نوسان‌هایی که مدام علامت عوض می‌کنند همدیگر را خنثی می‌کنند.
- مقدار رایج β برابر ۰/۹ است؛ مثل توپی که در سراشیبی می‌غلتد.

RMSprop: نرخ یادگیری جداگانه برای هر پارامتر

$$\mathbf{s}_t = \beta\, \mathbf{s}_{t-1} + (1-\beta)\, \mathbf{g}_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon}$$

- گرادیان بر ریشه میانگین مربعات گرادیان‌های اخیر تقسیم می‌شود.
- گرادیان بزرگ: قدم کوچک‌تر؛ گرادیان کوچک: قدم بزرگ‌تر.

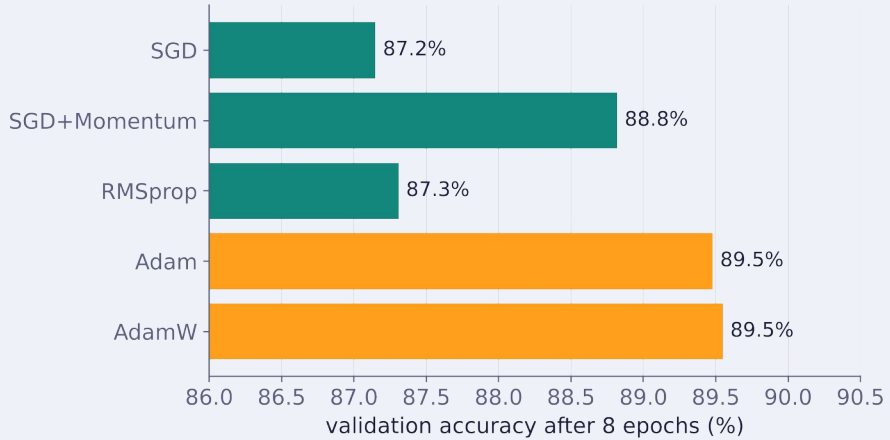
Adam = Momentum + RMSprop

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \qquad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t} \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

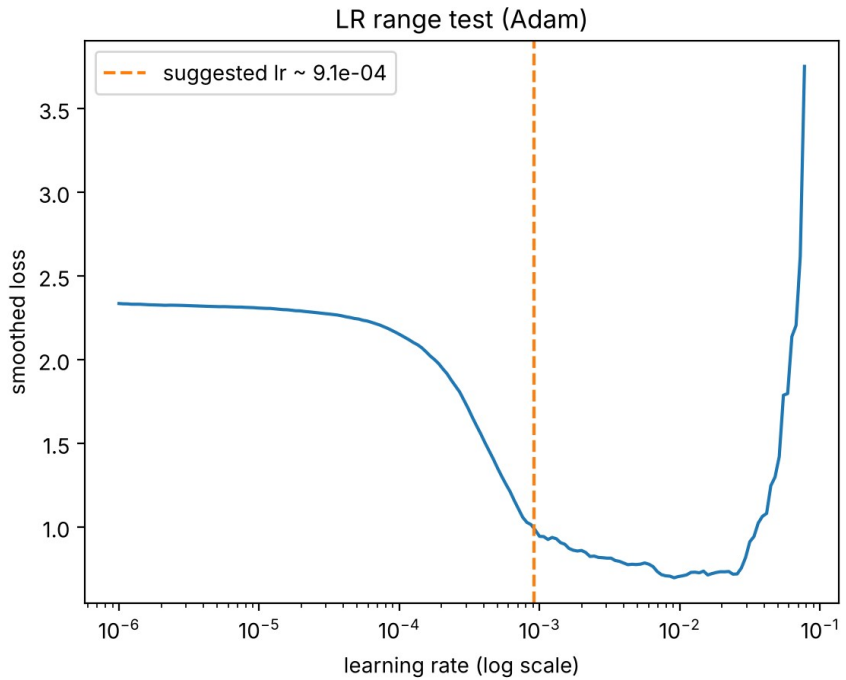
- تصحیح بایاس، شروع m و v از صفر را جبران می‌کند (Kingma & Ba, 2015).
- پیش‌فرض‌ها: $\beta_1 = 0.9$ و $\beta_2 = 0.999$ و نرخ یادگیری ۰/۰۰۱
- AdamW کاهش وزن را جدا از گرادیان اعمال می‌کند؛ استاندارد Transformerها.

مقایسه روی Fashion-MNIST



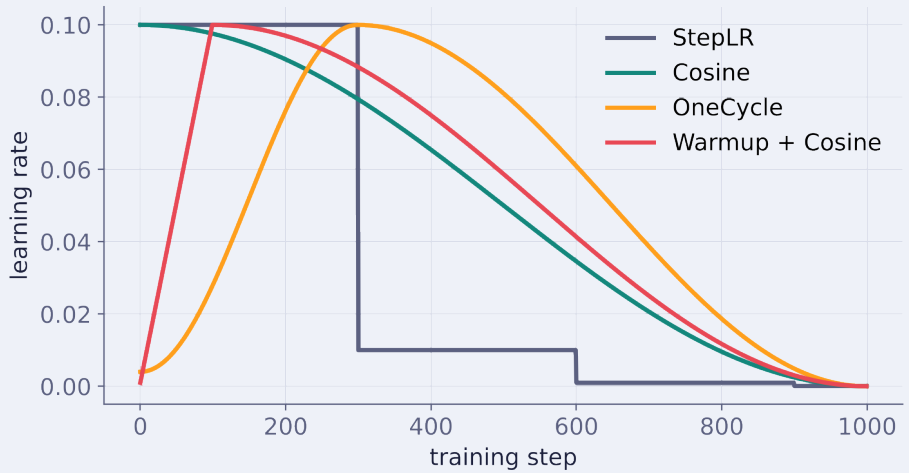
- همان MLP جلسه ۴، با وزن‌های اولیه یکسان و ۸ Epoch
- SGD ساده کندترین است؛ Momentum تفاوت بزرگی ایجاد می‌کند.
- Adam و AdamW بدون تنظیم زیاد بهترین نتیجه را دادند.
- نقطه شروع پیشنهادی: AdamW با نرخ یادگیری ۰/۰۰۱

LR Range Test: پیدا کردن نرخ یادگیری



- نرخ یادگیری را در هر Batch به‌صورت نمایی بزرگ می‌کنیم.
- تا جایی که Loss منفجر شود ادامه می‌دهیم.
- نرخ مناسب: حدود یک دهم نقطه کمینه Loss
- برای Adam روی این مسئله: حدود ۰/۰۰۱

زمان‌بندی نرخ یادگیری (LR Scheduler)



- ابتدا قدم‌های بزرگ، در پایان قدم‌های کوچک
- StepLR: کلاسیک برای CNN
- Cosine و OneCycle: رایج در بینایی ماشین
- Warmup + Cosine: استاندارد Transformerها
- با OneCycle بهترین Loss اعتبارسنجی از ۰/۳۰۵ به ۰/۲۸۰ رسید.

استفاده از Scheduler در PyTorch

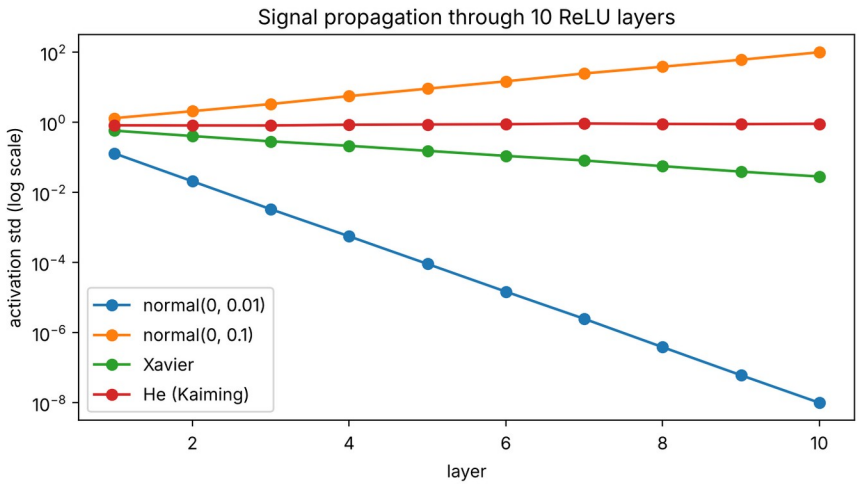
```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
                              weight_decay=1e-2)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=3e-3, epochs=EPOCHS,
    steps_per_epoch=len(train_loader))

for X, y in train_loader:
    loss = criterion(model(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    scheduler.step()        # per batch for OneCycleLR
```

- OneCycleLR و Warmup بر حسب گام تعریف می‌شوند: بعد از هر Batch.
- StepLR معمولاً بر حسب Epoch: در پایان هر Epoch.
- نرخ فعلی: optimizer.param_groups[0]['lr']

مقداردهی اولیه وزن‌ها

- همه صفر: مشکل تقارن؛ نورون‌ها هرگز متفاوت نمی‌شوند.
- خیلی کوچک: سیگنال محو می‌شود؛ خیلی بزرگ: منفجر.
- Xavier با واریانس 2/(n_in + n_out) برای Tanh
- He با واریانس n_in/2 برای ReLU
- پیش‌فرض PyTorch (Kaiming-Uniform) معمولاً کافی است.



جمع‌بندی

- Momentum نوسان را کم و حرکت در جهت‌های ثابت را تند می‌کند.
- RMSprop نرخ یادگیری هر پارامتر را تطبیقی می‌کند؛ Adam هر دو را ترکیب می‌کند.
- شروع پیشنهادی: AdamW با نرخ ۰/۰۰۱؛ برای CNN: SGD + Momentum + Cosine
- نرخ یادگیری را با LR Range Test پیدا کنید.
- از Scheduler استفاده کنید؛ Warmup برای Transformerها ضروری است.
- He برای ReLU و Xavier برای Tanh

تمرین

۱ تابع Rosenbrock را با چهار بهینه‌ساز پیاده‌سازی‌شده کمینه کنید و مسیرها را رسم کنید.

۲ Nesterov Momentum را اضافه کنید و تفاوتش را با Momentum معمولی توضیح دهید.

۳ Adam را با نرخ‌های ۰/۰۰۰۱، ۰/۰۰۱ و ۰/۰۱ آموزش دهید و با پیشنهاد LR Range Test مقایسه کنید.

۴ آزمایش مقداردهی را با Tanh تکرار کنید؛ کدام روش بهتر است؟

۵ SGD + Momentum را با StepLR آموزش دهید و scheduler را در پایان هر Epoch جلو ببرید.